# `scripts/check-challenge-discrimination.sh` — User Guide

**Last verified:** 2026-05-15 (1.2.23 closure-cycle, §6.AB-debt close) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate) + Lava §6.AB (Anti-Bluff Test-Suite Reinforcement, 27th §6.L invocation)

## Overview

Mechanical enforcement of §6.AB.3 — every `Challenge*Test.kt` file MUST carry a falsifiability rehearsal block in its KDoc that names a deliberately-broken-but-non-crashing version of the production code AND the assertion message the test produces against that mutation. Without this, a test that passes against today's HEAD has no proof it would CATCH a future non-crashing regression — i.e. it could be a §6.J-spirit bluff.

Closes §6.AB-debt from ❌ deferred to ✅ wired (no Detekt setup required — bash scanner consistent with §6.AC pattern).

## Two-layer enforcement (Layer 2 added 2026-05-15 from bluff-hunt audit)

### Layer 1: KDoc marker check

Any one of these in the test file's KDoc satisfies Layer 1:

1. `FALSIFIABILITY REHEARSAL` (canonical, used by C00–C29; optionally prefixed with `§6.AB.3`)
2. `§6.AB-discrimination:` block (alternate canonical form)
3. Companion file at `.lava-ci-evidence/sp3a-challenges/<TestName>-<sha>.json` with a `falsifiability_rehearsal` / `discrimination` / `bluff_classification` field

### Layer 2: BODY structural check

Files passing Layer 1 are also scanned for real-assertion patterns in the test body. A test that has the FALSIFIABILITY REHEARSAL marker but NO real assertion in its body is a §6.J spirit bluff: the doc claims discrimination, the body proves nothing.

Acceptable real-assertion patterns (any one is sufficient):

- Compose UI: `onNode|onAllNodes|assertIs|assertText|assertExists|fetchSemanticsNodes|composeRule\.waitUntil`
- JUnit assertions: `assertEquals|assertTrue|assertFalse|assertNotNull|assertSame|assertContains|assertThat`
- Classpath verification: `Class\.forName(...)` OR `::class\.java`
- Symbol references: `::<identifier>` (function-ref or class-ref proves the symbol exists at runtime)
- `check()` / `require()` calls

Falsifiability rehearsal (Layer 2):

- Mutation: synthetic `ChallengeBLUFF_REHEARSAL_DELETEMETest.kt` with the FALSIFIABILITY REHEARSAL block but no body assertions
- Observed: scanner reports "Layer 2 violations (marker present but body has no real assertion)" + names the file + lists remediation patterns
- Reverted: yes (synthetic file deleted; scanner returns to ✓ green)

Forensic anchor: `.lava-ci-evidence/bluff-hunt/2026-05-15-challenge-body-structural-audit.json` documented the manual audit that motivated this Layer 2 mechanical enforcement.

# Usage

```
bash scripts/check-challenge-discrimination.sh
```

Default mode is STRICT (exit 1 on any violation). Set `LAVA_CHALLENGE_DISCRIMINATION_STRICT=0` to run in advisory mode.

# Inputs

None (walks `app/src/androidTest/kotlin/lava/app/challenges/Challenge*Test.kt`).

# Outputs

- One stdout summary block + per-violation list
- Exit 0 if 0 violations OR advisory mode; exit 1 in strict mode with violations

# Side-effects

None — read-only scan.

# Falsifiability rehearsal of the scanner itself

Test: scanner fires when a Challenge test's marker is stripped

- Mutation: `sed 's/FALSIFIABILITY REHEARSAL/[STRIPPED]/g'` on `Challenge00CrashSurvivalTest.kt`
- Observed: scanner reports "failing on 1 violation(s)" + lists the file path
- Reverted: yes (file restored; scanner reports 0 violations again)

# Edge cases

## Companion file with empty discrimination field

The scanner only checks for the field NAME, not its content. A future enhancement could parse the JSON and verify the field is non-empty + names a real mutation. For now: shipping `{"falsifiability_rehearsal": ""}` would silently pass — counted as a known limitation, tracked as §6.AB rolling improvement.

## New Challenge test added without marker

Pre-push hook Layer 2 (`scripts/ci.sh --changed-only`) invokes this scanner. A commit that adds `app/src/androidTest/.../ChallengeXX_NoMarker.kt` will be REJECTED by pre-push under strict mode.

# Cross-references

- `scripts/check-non-fatal-coverage.sh` (sister scanner for §6.AC)
- `docs/helix-constitution-gates.md` (gate inventory)
- Lava `CLAUDE.md` §6.AB (the mandate)
- Lava `CLAUDE.md` §6.J (the parent anti-bluff principle)